Week 2 Provider Based LLM Client

-D. Mohit Venkat Sai

-2520030106

Aim

To build a provider based Python client that can send prompts to either a hosted Gemini model or a locally running Ollama model.

Objectives

- Define a common interface for different LLM providers.
- Generate a response using Ollama and the llama3.2 model.
- Configure Gemini as an alternative provider.
- Select a provider when the program runs.

Software and Requirements

- Visual Studio Code and Python
- A Python virtual environment
- Ollama with the llama3.2 model
- A Gemini API key for the Gemini option
- Python packages: python-dotenv, google-genai, and requests

System Design

The program separates provider-specific requests from the common client.

- **LLMProvider** defines the shared generate method.
- **OllamaProvider** sends prompts to the local Ollama service.
- **LLMClient** calls the selected provider through the shared interface.

This design allows the program to switch providers without changing how the client submits a prompt.

Procedure

1. Installed Ollama and downloaded the llama3.2 model.
2. Created a week-2 project folder in Visual Studio Code.
3. Activated the project's virtual environment.
4. Installed the required packages:

```
python -m pip install -r requirements.txt
```

5. Created llm_client.py with the shared provider interface, Gemini provider, Ollama provider, and common client.
6. Created main.py to load the API key, ask the user to choose a provider, accept a prompt, and print the response.
7. Added the Gemini key to .env for use with the Gemini option.
8. Ran the program:

python main.py

9. Selected ollama, entered a prompt, and received a response from the local model.

Provider Interface

The common interface lets both providers use the same method name:

```
from abc import ABC, abstractmethod


class LLMProvider(ABC):
  @abstractmethod
  def generate(self, prompt):
    pass
```

The Ollama provider sends a request to the local Ollama service:

```
import requests


class OllamaProvider(LLMProvider):
  def __init__(self, model="llama3.2"):
    self.model = model
    self.url = "http://localhost:11434/api/generate"


  def generate(self, prompt):
    response = requests.post(
      self.url,
      json={
        "model": self.model,
        "prompt": prompt,
        "stream": False,
      },
```

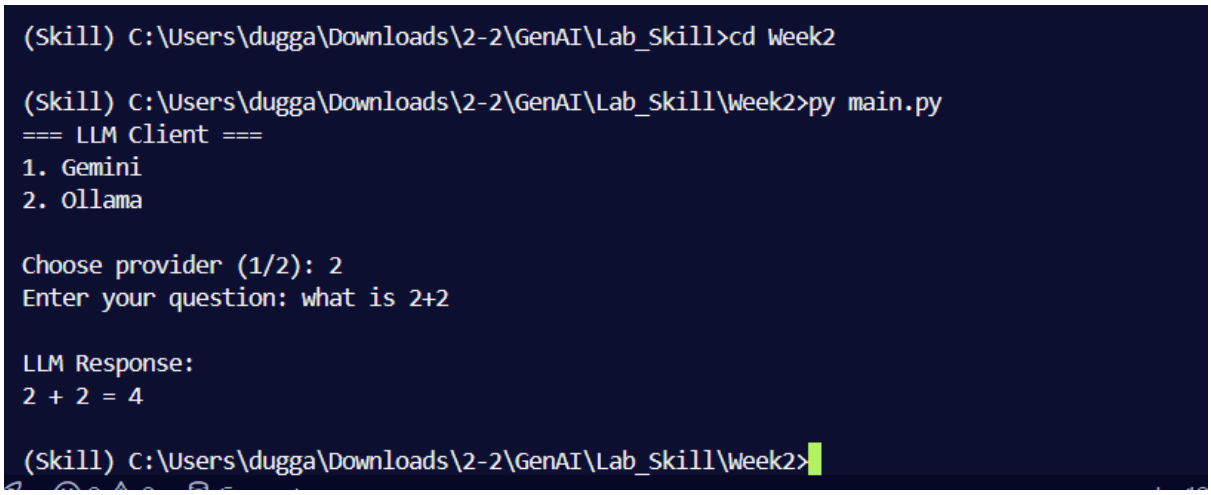
```
        timeout=120,
    )
    response.raise_for_status()
    return response.json()["response"]
```

The common client delegates prompt generation to the selected provider:

```
class LLMClient:
  def __init__(self, provider):
    self.provider = provider

  def generate(self, prompt):
    return self.provider.generate(prompt)
```

Result and Observation



The program displayed a provider selection prompt. The Ollama option successfully generated a response using llama3.2.

The Gemini option was included in the program. The Week 2 run documented here verified Ollama; Gemini was tested separately during the Week 3 experiment.

Conclusion

The experiment successfully demonstrated a provider based LLM client. A shared interface allowed the same client to work with the local Ollama provider, while keeping provider-specific request handling inside each provider class. The Ollama response confirmed that the local model connection was working.